

SAST Lab 4 – OWASP Mutillidae II & Custom Semgrep Rules

Name: Abdelrahman Sameh Kamal

ID: 2305039

The two main vulnerabilities analysed in this report are:

1. A **SQL Injection** vulnerability in the “View Account Details” feature.
2. A **Reflected Cross-Site Scripting (XSS)** vulnerability in an input/echo page.

Vulnerability 1 – SQL Injection in “View Account Details”

Description and Impact

The first vulnerability is a **SQL Injection** in the feature that displays account details based on a supplied username and password. The page asks the user to “Please enter username and password to view account details.”

The form accepts Name and Password. Instead of a normal password, the following payload was used for the password field: (' OR 1=1 --)

The resulting HTTP request contains parameters like:

```
page=user-info.php&username=admin&password=' OR 1=1 --&user-info-php-submit-button=View+Account+Details
```

When this request is processed, Mutillidae constructs a SQL query similar to:

```
SELECT * FROM accounts WHERE username='admin' AND password=' OR 1=1 --'
```

The application then shows a detailed error page generated by MySQLHandler.php:

- File: /app/classes/MySQLHandler.php
- Message: MySQL syntax error around the injected string
- Query: SELECT * FROM accounts WHERE username='admin' AND password=' OR 1=1 --'



The presence of attacker-controlled input inside the SQL statement confirms the SQL Injection vulnerability.

Root Cause in Source Code:

From the stack trace and source exploration, the account retrieval logic flows through:

1. user-info.php – reads username and password from the HTTP request and calls the query handler.
2. **classes/SQLQueryHandler.php** builds the SQL string using raw input.

```
if ($!formSubmitted){
    if ($!ProtectAgainstMethodTampering) {
        $!userInfoSubmitButton = $_POST["user-info-php-submit-button"];
        $!username = $_POST["username"];
        $!password = $_POST["password"];
    }else{
        $!userInfoSubmitButton = $_REQUEST["user-info-php-submit-button"];
        $!username = $_REQUEST["username"];
        $!password = $_REQUEST["password"];
    }
    // end if $!ProtectAgainstMethodTampering
}
// end if $!formSubmitted
```

```
0 references [0] overridden
public function getUserAccount($pUsername, $pPassword): mixed{
    /*
     * Note: While escaping works ok in some case, it is not the best defense.
     * Using stored procedures is a much stronger defense.
     */

    if ($this->stopSQLInjection){
        $pUsername = $this->MySQLHandler->escapeDangerousCharacters($pUsername);
        $pPassword = $this->MySQLHandler->escapeDangerousCharacters($pPassword);
    }
    // end if

    $!queryString =
        "SELECT * FROM accounts
        WHERE username='".$pUsername.
        "' AND password='".$pPassword."'";

    return $this->MySQLHandler->executeQuery($!queryString);
}
//end public function getUserAccount
```

3. `classes/MySQLHandler.php` executes the query on the database.

```
28 references | 0 overrides  
public function executeQuery($pQueryString): mixed{  
    return $this->doExecuteQuery(pQueryString: $pQueryString);  
}// end public function executeQuery
```

The key problems are:

- Untrusted data (\$username, \$password) are **directly concatenated** into the SQL string.
- No parameterized queries or escaping are used.
- The query is executed as-is against the database.

Mitigation Recommendations:

To mitigate this SQL Injection, the following changes are recommended:

- SQL queries.
- Validate and sanitize user input on the server side.
- Avoid displaying detailed SQL error messages in production.
- Use least-privilege accounts for the database.

Vulnerability 2 – Reflected Reflected XSS in DNS Lookup

Description and Impact

The second vulnerability is a Reflected Cross-Site Scripting (XSS) issue. On a specific page, the application reads an HTTP GET parameter and echoes it directly back into the HTML response without any encoding.

An attacker can craft a malicious link that includes a `<script>` payload. When a victim clicks the link, the script executes in the context of the Mutillidae domain

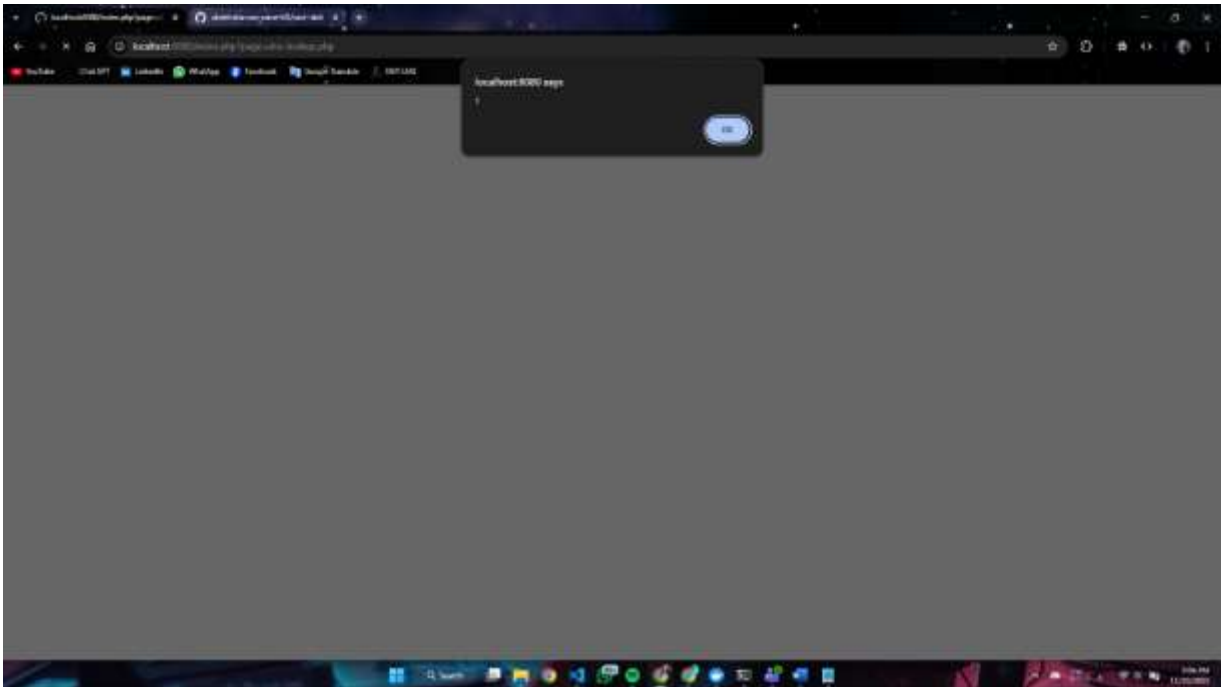
The page takes a parameter such as message. When accessed with the following URL:

`http://localhost:8080/index.php?page=dns-lookup.php&host=<script>alert(1)</script>`

the application returns a response that renders an alert dialog (`alert(1)`), confirming that the JavaScript payload is executed in the browser. This demonstrates a reflected XSS vulnerability

Payload:

`<script>alert(1)</script>`



Root Cause in Source Code:

1. This code takes the value from the `target_host` field and saves it into `$_lTargetHost` without any cleaning or validation.

```
$_lFormSubmitted = false;
if (isset($_POST["target_host"]) || isset($_REQUEST["target_host"])) {
    $_lFormSubmitted = true;
} // end if

if ($_lFormSubmitted){
    $_lProtectAgainstMethodTampering?$_lTargetHost = $_POST["target_host"]:_lTargetHost = $_REQUEST["target_host"];
```

2. This code turns XSS protection on or off. When protection is disabled, `$_lTargetHostText` is set to the raw user input without HTML encoding, so XSS becomes possible.

```
if ($_lProtectAgainstXSS) {
    /* Protect against XSS by output encoding */
    $_lTargetHostText = $Encoder->encodeForHTML($_lTargetHost);
}else{
    $_lTargetHostText = $_lTargetHost;           //allow XSS by not encoding output
} //end if
```

3. This code prints `$_lTargetHostText` directly inside the HTML page. If it contains `<script>...</script>`, the browser will execute it (XSS).

```
if ($_lFormSubmitted){
    try{
        if ($_lTargetHostValidated){
            echo '<div class="report-header">Results for ' . $_lTargetHostText . '</div>';
```

The key problems are:

- The value of target_host comes from \$_POST / \$_REQUEST and is stored in \$lTargetHost without any sanitization.
- When XSS protection is disabled, \$lTargetHostText is set to the raw user input (no HTML encoding).
- The page relies on client-side JavaScript validation, but there is no strong server-side protection, so a malicious <script> payload can still be reflected back and executed.

Mitigation Recommendations:

- Always apply **HTML output encoding** (htmlspecialchars() in PHP) before printing user input into the page.
- Keep XSS protection enabled for all security levels in a real application; do not allow raw output of user data.
- Consider enabling a Content Security Policy (CSP) to reduce XSS impact.

Semgrep Rules + Tests:

To detect the vulnerable code patterns in **Mutillidae**, I created two custom **Semgrep** rules and a small set of test files. The goal was to automatically find the same SQL injection and XSS issues that I reproduced manually in the application.

Rule 1 – SQL Injection (php-sqli.yaml):

This rule is for SQL injection.

It finds places where user input is added into a SQL query string and sent to mysql_query().

```
rules > php-sqli.yaml
1 rules:
2   - id: mutillidae-php_sqli-unsafe-concat
3     languages: [php]
4     severity: HIGH
5     message: >
6       Possible SQL injection: mysql_query is called with an argument
7       that concatenates user input into the SQL string.
8     metadata:
9       category: security
10      technology: [php, mysql]
11      cwe: ["CWE-89: SQL Injection"]
12      owasp: ["A03:2021 - Injection"]
13    pattern-either:
14      - pattern: |
15        mysql_query("..." . $_GET["..."] . "...");
16      - pattern: |
17        mysql_query("..." . $_POST["..."] . "...");
18      - pattern: |
19        mysql_query("..." . $_REQUEST["..."] . "...");
```

Rule 2 – DNS Lookup (php-xss-dns.yaml):

This rule is for reflected XSS in dns-lookup.php.

It finds \$_REQUEST data that is printed inside HTML without htmlspecialchars().

```
rules > cat php-xss-dns.yaml
1 ruleid:
2 id: mutillidae-php-xss-echo
3 languages: [php]
4 severity: HIGH
5 message: >
6   Possible reflected XSS: data from $_GET/$_POST/$_REQUEST is echoed
7   directly without HTML encoding.
8 metadata:
9   category: security
10  technology: [php]
11  cwe: ["CWE-79: Cross-site Scripting"]
12  owasp: ["A03:2021 - Injection/XSS"]
13 pattern-either:
14   - pattern: echo $_GET["..."];
15   - pattern: echo $_POST["..."];
16   - pattern: echo $_REQUEST["..."];
17
```

Test Rule 1 – SQLi:

Here I test the SQLi rule.

The first code is unsafe (Semgrep should catch it).

The second code is safe (Semgrep should ignore it).

```
rules > cat php-sqli.php
1 <?php
2 // ruleid: mutillidae-php-sqli-unsafe-concat
3 mysql_query(query: "SELECT * FROM accounts WHERE user = '" . $_GET['username'] . "'");
4
5 // ok: mutillidae-php-sqli-unsafe-concat
6 $user = $_GET['username'];
7 $stmt = $pdo->prepare('SELECT * FROM accounts WHERE user = ?');
8 $stmt->execute([$user]);
```

Test Rule 2 – XSS (php-xss-dns.php):

Here I test the XSS rule.

The first echo is unsafe.

The second echo is encoded and safe.

```
rules > cat php-xss-dns.php
1 <?php
2 // ruleid: mutillidae-php-xss-echo
3 echo $_REQUEST['target_host'];
4
5 // ok: mutillidae-php-xss-echo
6 $safe = htmlspecialchars(string: $_REQUEST['target_host'], flags: ENT_QUOTES, encoding: 'UTF-8');
7 echo $safe;
8
```

Semgrep –test:

```
PS C:\Users\Lenovo\Documents\semster 5\Secure Software\sast-lab4> docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep --test /src/semgrep/rules
```

Scan Status
Scanning 1 file with 1 Code rule:
Scanning 1 file.

Scan Status
Scanning 1 file with 1 Code rule:
Scanning 1 file.
2/2: ✓ All tests passed

Running semgrep --test on my custom rules.

Semgrep reports “2/2: All tests passed”, which means both the SQLi and XSS rules correctly detect the vulnerable and safe examples.

Main Scan for Mutillidae:

Semgrep scan on the Mutillidae PHP source code using my custom rules.

The findings show locations of SQL injection and XSS that match the vulnerabilities described in the report.

```
PS C:\Users\Lenovo\Documents\semster 5\Secure Software\sast-lab4> docker run --rm -v "${PWD}:/src" semgrep/semgrep semgrep '
>> --config /src/semgrep/rules '
>> /src/mutillidae/src
```

Scan Status
Scanning 392 files tracked by git with 2 Code rules:
Scanning 178 files with 2 php rules.

1 Code Finding

/src/mutillidae/src/source-viewer.php
>>> semgrep.rules.mutillidae-php-xss-echo
Possible reflected XSS: data from \$_GET/\$_POST/\$_REQUEST is echoed directly without HTML encoding.

81: <input type="hidden" name="page" value="<?php echo \$_REQUEST['page'];?>">

Scan Summary
✔ Scan completed successfully.
• Findings: 1 (1 blocking)
• Rules run: 2
• Targets scanned: 178
• Parsed lines: ~99.9%
• Scan skipped:
 • Files larger than files 1.0 MB: 1
 • Files matching .semgrepignore patterns: 2
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 2 rules on 178 files: 1 finding.